Hochschule Konstanz
Fakultät Informatik

Software Architecture

Introduction to Software Architecture

Prof. Dr. Marko Boger

Before we begin

- Language
  - Slides are in English
  - Presentation is in German, unless an english speaking student requests English
- Infrastructure
  - Moodle
    - Register for this Lecture in Moodle
  - Git/Github
  - Discord
  - sbt
  - Coveralls
- Organization
  - Lecture
    - Wednesday 08:00 – 9:30
  - Exercise
    - Wednesday 9:45 – 11:15
    - Discord
  - Grade is determined by
    - final presentation and
    - semester tasks

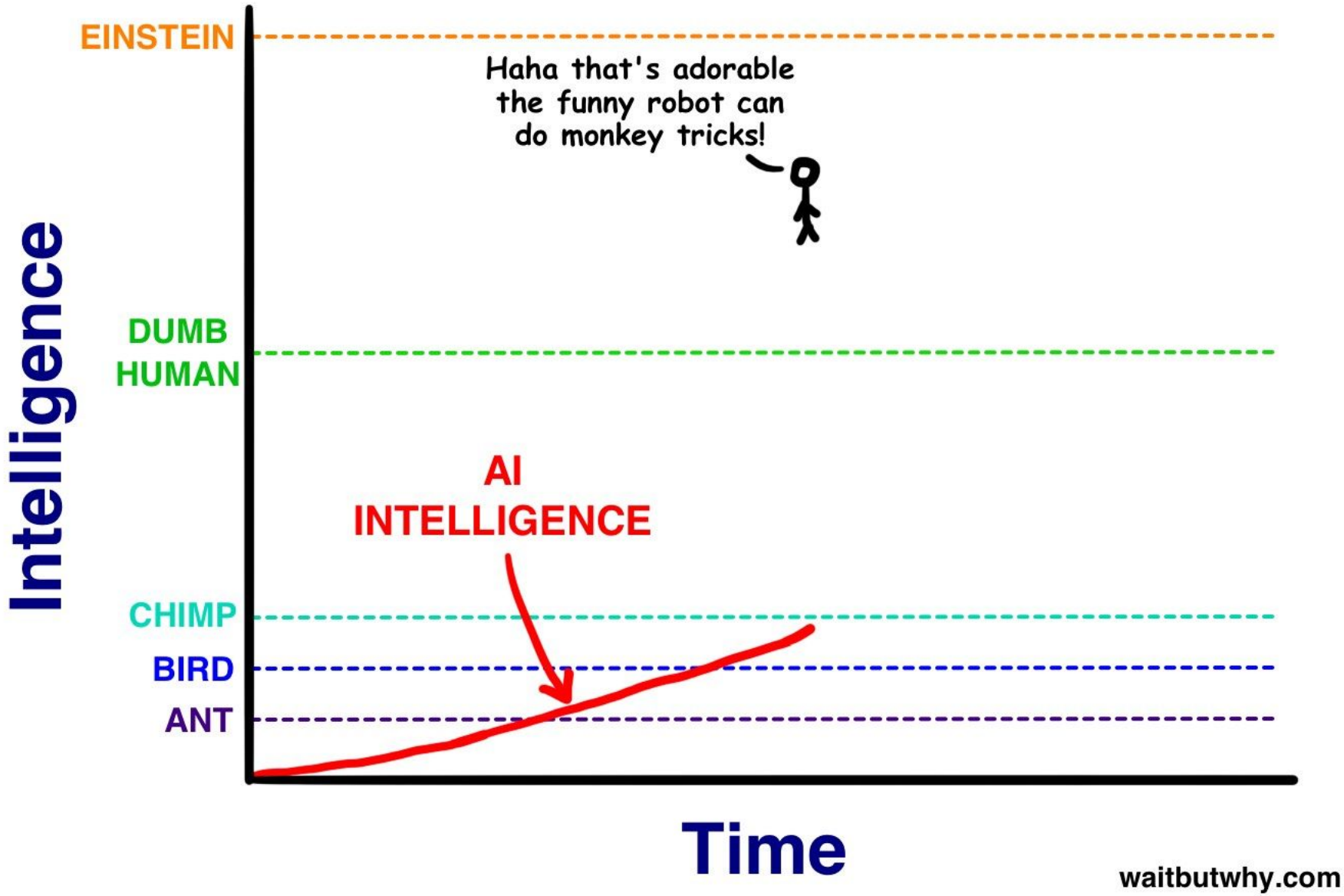
Goal of this Lecture

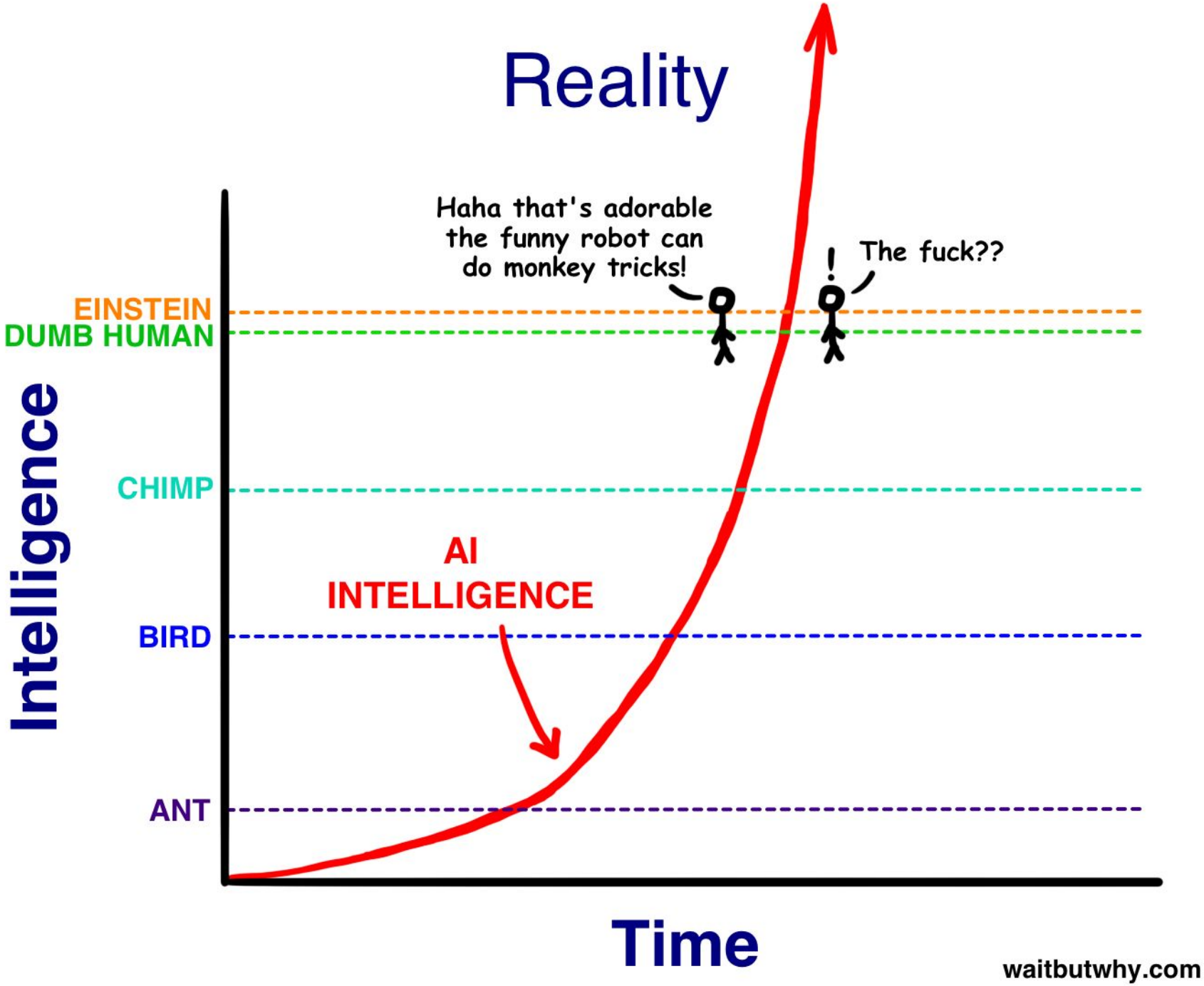
- Understand the role of the software architect in a team
  - The ability to communicate about architectural aspects of software
  - The ability to foresee architectural requirements
  - The ability to restrict to current architectural requirements
  - The ability to find the compromise between these two
- Turn a project in to a product
  - Maintainability
  - Extensibility
  - Durability
  - Scalability
  - Robustness
  - Performance

Your task for this Lecture

- Take an existing Game
  - Developed as in Software Engineering and Web-Applications
  - MVC, Components, DI, tests
  - TUI, GUI, WUI
- Componentise this
  - 2 interchangeable database technologies
  - Layers and slices
- Exchange the communication mechanism from synchronous to asynchronous
- Exchange deployment from monolithic to distributed
- Test and increase robustness, scalability

Our Distorted View of Intelligence




Reality
Haha that's adorable the funny robot can do monkey tricks!
The fuck??
EINSTEIN
DUMB HUMAN
CHIMP
AI INTELLIGENCE
BIRD
ANT
Intelligence
Time
waitbutwhy.com

Focus on AI-driven development

We will use AI-Tools to develop scalable software architectures

- explore functional programming from a masterpiece of software
- explore different architectural styles
- from desktop to big data

Introducing Lichess

Lichess is an open source chess application, written in Scala

- created by Thibault Duplessis (FR, 1984)
- created in 2010, grew during Covid, second biggest platform
- >5 million games played per day
- billions of games in database

https://www.youtube.com/watch?v=7VSVfQcaxFY

How 1 Software Engineer Outperforms 138 - Lichess Case Study

390K views • 1 year ago

Tom Delalande

This is a case study on the **Lichess** product. A free and open source chess platform with over 4 million monthly active users and 1 ...

13 chapters Intro | Story to kill retention | Lichess vs Chess.com | Scala | Play Framework | MongoDB | Snabbdom |...

Plan for this lecture series

1. Monolith, Testsuite, TextUI
2. Functional style, chess rules
3. Export and Import
   a. JSON, Circe
   b. PGN, FEN, ParserCombinators
4. HTTP access
   a. http4s
   b. fs2
5. Microservices
6. Persistence
   a. MongoDB,
   b. PostgreSQL
7. Web-UI
8. Performance testing
   a. Gatlin
   b. JMH
9. Bot API
10. Reactive Streams
11. Kafka
12. Spark
13. Tournament
14. Presentation

Tools

IDEs

- Cursor
- Github Copilot
- Claude Code
- Jetbrains IntelliJ
- Windsurf
- Antigravity

Testsuite and Build automation

- Scala fmt
- sbt
- ScalaTest
  - 100% code coverage
- Scoverage
- GitHub
-

Rules for the Lecture

- Use AI heavily!
  - The goal is to learn faster, not to jump to solutions.
- Less Lecturing, more interaction
  - Two groups present their results during every class
    - one group by upfront assignment
    - one group by merrit

Programming Chess

Coding Adventures:Chess by Sebastian Lague

- Introduction into the rules and complexities of Chess
- https://www.youtube.com/watch?v=U4ogK0MIzqk

Task

Create a simple chess game.

- Not all chess rules need to be implemented (yet).
- Simple text UI on console.
- You can make moves, they do not have to be legal yet.
- Use AI intensively.
  - pure Scala
  - Functional style
- Generate Tests
  - 100% code coverage

What is Software Architecture?

- Architecture is a set of principal design decisions about a software system
  - Every application has an architecture
    - It may be implicit
    - It may be a bad architecture
    - But it is good to make it explicit and turn it into a good architecture
  - Architecture is not a phase of development
    - Architecture can be planned before design or development
    - But it can not be finished at that time
    - Architecture is an ongoing process
- "Architecture is the art of making the arrows point in the right direction", Robert Martin

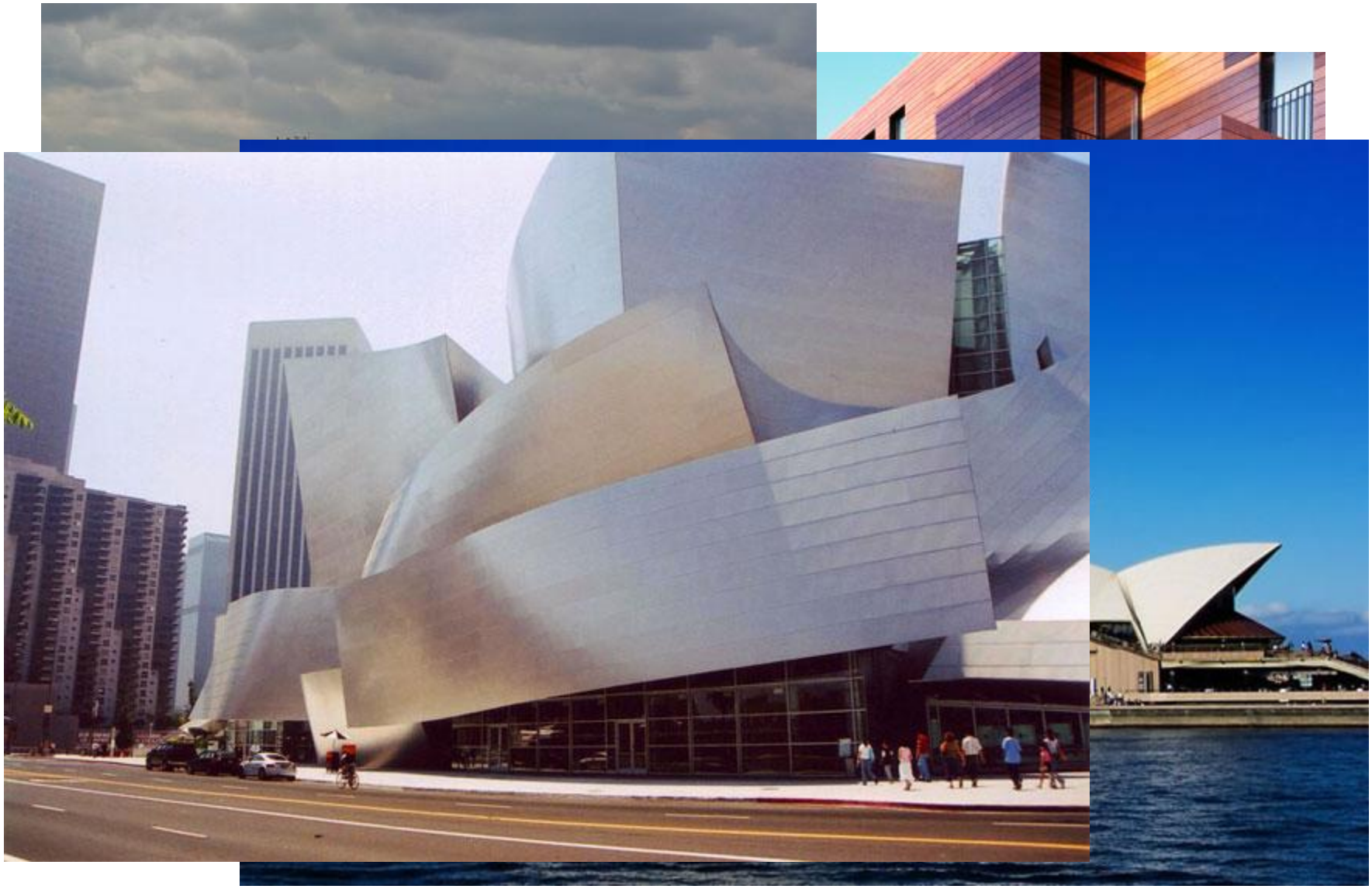
What influences the Architecture?

- Functional Requirements
- Non-Functional Requirements
- Financial restrictions
- Experience
- Available technology
- Existing implementations
- Available libraries, components or frameworks

Architecture Process

- The process of developing an architecture is iterative
  - The up-front architecture design can only be a starting point
- Our understanding of what works, and how it works, affects our perceived needs
- The insights from our experiences with existing systems
  - helps us imagine what might work and
  - enables us to assess development time and costs

What is Architecture?



What is Architecture?

The Importance of Architecture

- Software architecture forms the backbone for building successful software systems. An architecture largely permits or precludes a system's quality attributes such as performance or reliability. Architecture represents a capitalized investment, an abstract reusable model that can be transferred from one system to the next. The right architecture is the linchpin for software project success. The wrong one is a recipe for disaster.

Carnegie Mellon
Software Engineering Institute

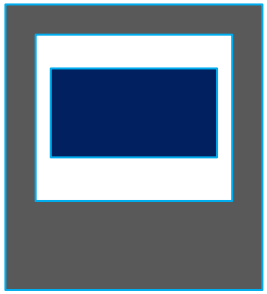
Evolution of Architectures

Scope of Software Engineering

- Monolyth, Ball of Mud

- Extraction of reusable assests

Library



Framework

- Layered Architecture

- Component-based Architecture

Evolution of Architectures

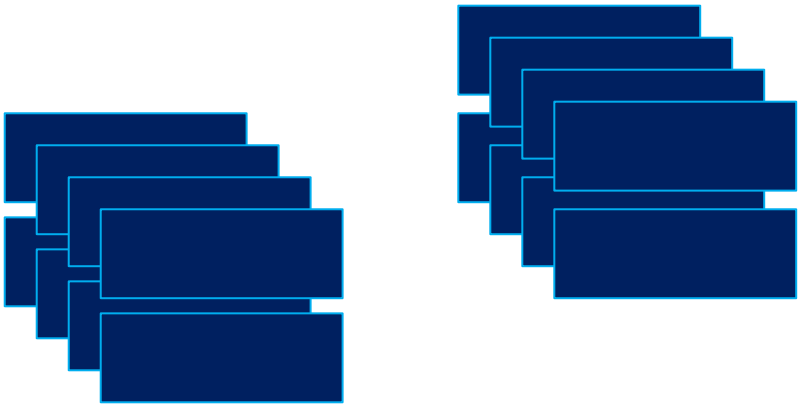
Scope of this lecture

- Extendable Architecture
  - Plug-in mechanism

- Distributed Architecture

- Persistent Architecture

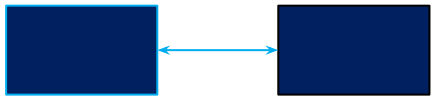
- Scalable Architecture



Evolution of Architectures

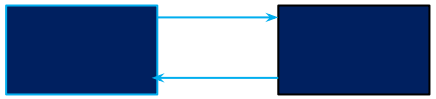
Scope of this lecture

- Sychronous/Local
  - Plug-in mechanism

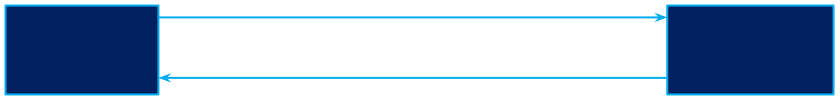


- Synchronous/Remote

- Asynchronous/Local



- Asynchronous/Remote

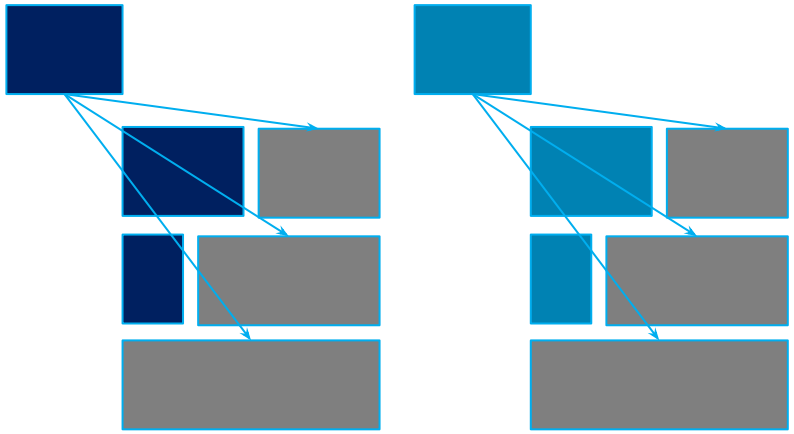


Evolution of Architectures

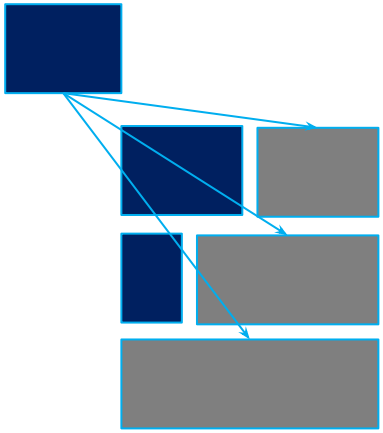
Scope beyond this lecture

- Product Line

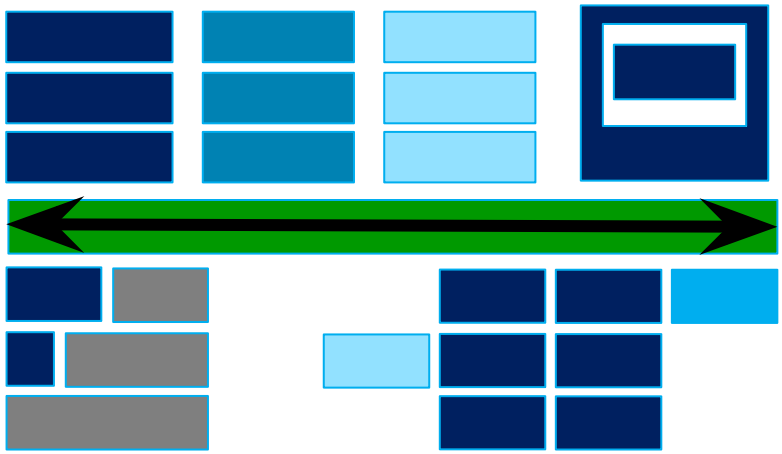
- Product Family



- Generative Architecture



- Enterprise Architecture



Product Line

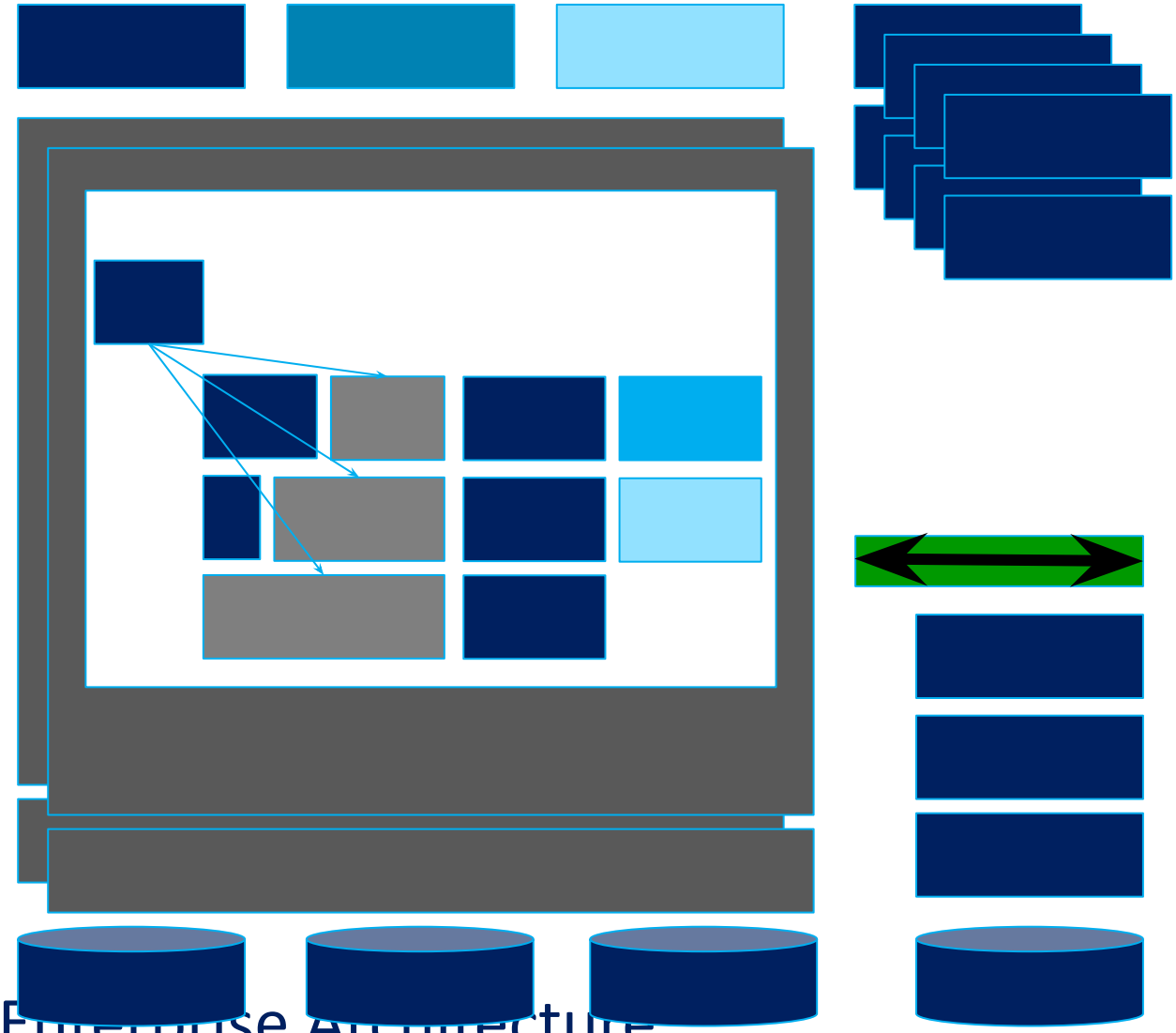
- Same application, but
  - Different hardware or
  - Different feature set or
  - Different user interface or
  - Different backend
- Caused by
  - Different kinds of customer
  - Different pricemodel
  - Different legal restrictions
- Examples
  - UML modeling tool
    - Differerent Editions for a different price
  - Heater Controll software
    - Runs on different hardware
- Techniques
  - Compiler directives
  - Components
  - Plugins
  - Generative
  - Variation points

Product Family

- Similar application
  - Similar domain
  - Similar process
  - Similar interface
  - Similar technology
- But
  - Different data sets
  - Different abstractions
- Caused by
  - Productivity gains if the similarities can be factored out and reused
- Examples
  - Web system for banking and insurance industry
    - Every customer has a different set of abstractions and processes
    - But the general architecture is the same
- Techniques
  - The difference can be modeled
  - The commonality can be extracted into a framework
  - Generation from model

Combination of Architectural Styles

- Libraries
- Frameworks
- Layers
- Components
- Generative
- Distributed
- Persistent
- Scalable
- Product Line
  - Multiple UIs
- Embedded in an Enterprise Architecture

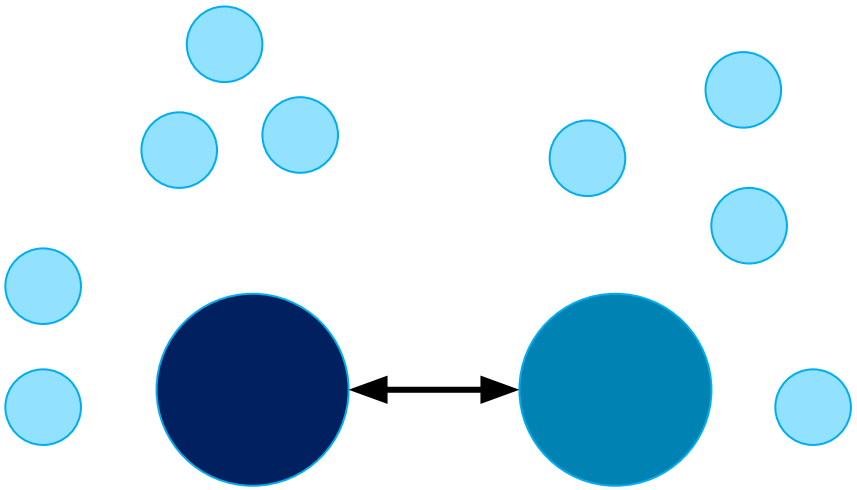


What does a Software Architect do?

- Understand the given architecture
- Analyse the requirements and constraints for the next phase of development
- Select languages, tools, components to be used
- Develop a target architecture for the next phase of development
- Manage the transition
- Document the architecture

Structure of a Team

- Many software development teams are organized around two central roles
  - Project Manager
  - Software Architect
- The interaction between these is intense
- Other team members organize around these
  - Developer, test engineer, deployment specialist, performance specialist, documentation engineer, ...



Role of the Project Manager

- Organize the development process
- Monitor progress
- Control cost
- Ensure deadlines
- Hire staff
- Communicate to upper management
- Communicate to customers management

Role of the Architect

- Responsible for the technical quality of code and architecture
- Communicates to customers domain experts
- Understands requirements of the domain and limitations of technology
- Knows and evaluates many available technologies to select the most appropriate ones
- Translates domain knowledge to developers
- Estimates costs
- Assigns tasks to other developers
- Controls and assists with technical progress of tasks

Case Study: Poseidon

- Based on the open source project ArgoUML
  - Big ball of mud
  - No tests
  - Large refactorings needed without the support of a test suite
- Introduced an external, generated repository
  - Library
  - Generative
- Introduced Community, Standard and Professional Edition
  - Product line
  - Components
  - Plug-in architecture

Case Study: Poseidon II

- Data im- and export to several formats
  - Enterprise Architecture
- Introduced Enterprise Edition
  - Distributed multi-user environment with central server
- Restructured the entire application to generate large parts of the code to support DSLs and other notations
  - Framework, Generative

Case Study: SeaPal

- Started out as simple logbook project on the iPhone
  - Layer
- Then it became a product
  - Components
- In-App purchases as plug-ins
  - Plugin-architecture
- Server backend to store data in the cloud
  - Distributed, Persistent
- The server data should also be presented in the web
  - Multiple Uis, Multi-User

Case Study: SeaPal II

- Support for Android
  - Product Line, multiple products
- We import map and harbor information from various data sources
  - USA map, Brazil maps, New Zealand maps etc.
  - Distributed, persistent
- We built an adapter to the SAP Sailing Analytics tool
  - Enterprise Architecture
- When the server synchronization is fully deployed we expect scalability issues
  - Scalable

Summary

- Software evolves
  - Similar to cities
- Software Architecture is not an up-front and fixed design
  - Software architecture has to be planed for change
- The software architect develops these architectures and manages the change over time
  - Transition from architectural style to architectural style